Configuracion de Git en Windows

Instalacion + SSH + GitHub | Paso a paso

El proceso completo en 5 pasos:

1	**Instalar Git** Descargar e instalar desde git-scm.com, asegurarse de tildar Git Bash.
2	**Configurar identidad** Decirle a Git tu nombre y email con git config --global.
3	**Generar clave SSH** Crear el par de claves privada/publica con ssh-keygen.
4	**Agregar clave en GitHub** Pegar la clave publica en Settings → SSH keys de GitHub.
5	**Verificar conexión** Confirmar que todo funciona con ssh -T git@github.com.

Paso 1 — Instalar Git

Git es el sistema de control de versiones que vamos a usar. En Windows viene con Git Bash incluido, que es la terminal que usaremos para todos los comandos.

Descargar

1. Abrir el navegador e ir a: https://git-scm.com/install/windows
2. Se descarga automáticamente el instalador para Windows de 64 bits
3. Abrir el archivo descargado (.exe)

Instalar — opciones importantes

Durante la instalación aparecen varias pantallas. La mayoría se dejan por defecto, pero hay que prestar atención a estas:

Pantalla: Select Components

> **TILDAR:** Git Bash Here (permite abrir Git Bash desde cualquier carpeta con clic derecho)

Pantalla: Choosing the default editor

Seleccionar: Use Visual Studio Code as Git's default editor

Pantalla: Adjusting the name of the initial branch

Seleccionar: Override the default branch name → escribir: main

Pantalla: Adjusting your PATH environment

Seleccionar: Git from the command line and also from 3rd-party software (opcion del medio, recomendada)

Es una opción que le dice a Windows **desde dónde se puede usar Git**.

PATH es una variable de Windows que contiene la lista de carpetas donde el sistema busca programas. Cuando escribís git en una terminal, Windows busca en esa lista para encontrar el programa.

Las tres opciones que aparecen son:

Opción	Qué significa
Use Git from Git Bash only	Git solo funciona dentro de Git Bash. No funciona en CMD ni en PowerShell.
Git from the command line and also from 3rd-party software ☑	Git funciona en Git Bash, CMD, PowerShell y VSCode. **Esta es la recomendada.**
Use Git and optional Unix tools from Command Prompt	Instala herramientas extra de Linux en Windows. Puede pisar comandos de Windows. No recomendada.

En la práctica, elegir la del medio significa que podés escribir git status tanto en Git Bash como en la terminal integrada de VSCode o en el CMD, sin problemas.

Si elegís solo Git Bash, cada vez que abras la terminal de VSCode y escribas git, te diría que no reconoce el comando.

Resto de pantallas

Dejar todas las demás opciones por defecto y hacer clic en Next hasta llegar a Install.

Verificar la instalacion

Al terminar, abrir Git Bash (clic derecho en el escritorio → Git Bash Here) y ejecutar:

```
git --version
# Debe mostrar algo como: git version 2.44.0
```

Paso 2 — Configurar la identidad

Esta configuración le dice a Git quién sos. Aparece en cada commit que hagas. Se hace una sola vez por computadora.

> **IMPORTANTE:** Podés ejecutar estos comandos desde cualquier carpeta. El flag --global los aplica a toda la computadora, no a un proyecto específico.

Nombre de usuario

Reemplaza Tu Nombre con tu nombre real (o el que quieras que aparezca en los commits):

```
git config --global user.name "Tu Nombre"
```

Email

Usa el mismo email con el que te registraste en GitHub:

```
git config --global user.email "tu@email.com"
```

Editor por defecto

Para que Git abra VSCode cuando necesite que escribas algo:

```
git config --global core.editor "code --wait"
```

Verificar la configuración

Para confirmar que todo quedo guardado correctamente:

```
git config --list

# Debe mostrar entre otras cosas:
# user.name=Tu Nombre
# user.email=tu@email.com
```

Paso 3 — Generar la clave SSH

Que es SSH y para qué sirve

SSH es un sistema de autenticacion seguro. En lugar de escribir usuario y contraseña cada vez que subís código a GitHub, tu computadora y GitHub se reconocen automáticamente mediante un par de claves:

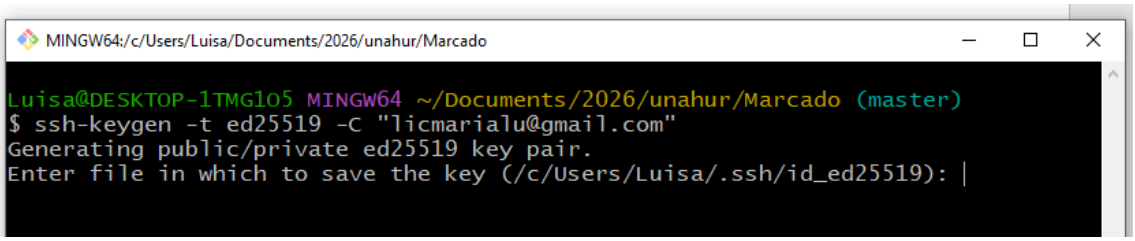
id_ed25519 CLAVE PRIVADA	Queda guardada en tu PC. Nunca la compartas ni la mandes a nadie.
id_ed25519.pub CLAVE PUBLICA	Esta la pegas en GitHub. Es seguro compartirla.

Generar el par de claves

Abre Git Bash y ejecuta (reemplaza el email por el tuyo de GitHub):

```
ssh-keygen -t ed25519 -C "tu@email.com"
```

Ejemplo:



Pregunta	Que hacer
`Enter file in which to save the key`	Presionar Enter (acepta la ubicación por defecto)
`Enter passphrase (empty for no passphrase)`	Presionar Enter (sin contraseña de clave)
`Enter same passphrase again`	Presionar Enter (confirmar sin contraseña)

Al finalizar, aparece un mensaje confirmando que los archivos fueron creados y un dibujo ASCII de la clave. Eso es normal.

Verificar que se crearon los archivos

```
ls ~/.ssh

# Debe mostrar:
# id_ed25519
# id_ed25519.pub
```

Copiar la clave pública

Este comando copia la clave al portapapeles (como Ctrl+C). No muestra nada en pantalla:

```
cat ~/.ssh/id_ed25519.pub | clip
```

Si querés ver el contenido antes de copiar:

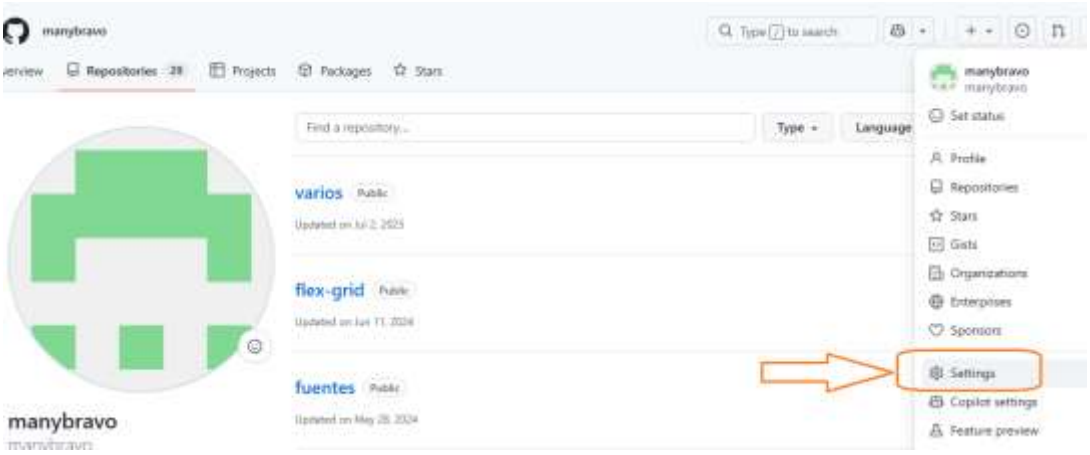
```
cat ~/.ssh/id_ed25519.pub

# Muestra una sola linea larga que empieza con:
# ssh-ed25519 AAAAC3Nz... tu@email.com
```

Paso 4 — Agregar la clave en GitHub

Ahora le decimos a GitHub que reconozca esta computadora. Para eso pegamos la clave pública en la configuración de nuestra cuenta.

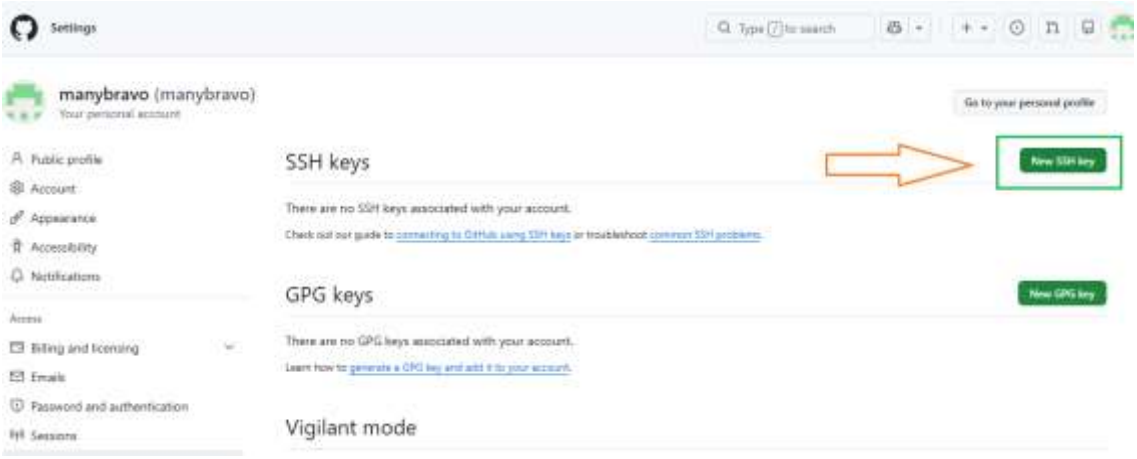
4. Abrir https://github.com e iniciar sesión
5. Hacer clic en la foto de perfil (arriba a la derecha) → Settings

6.



7. En el menú de la izquierda: SSH and GPG keys

8. Hacer clic en el botón verde: New SSH key



9. Completar el formulario:

Campo	Qué poner
Title	Un nombre para identificar la PC. Ej: PC casa, Laptop trabajo, Notebook aula 3
Key type	Dejar Authentication Key (ya viene seleccionado por defecto)
Key	Pegar con Ctrl+V la clave que copiaste en el paso anterior

10. Hacer clic en Add SSH key
11. GitHub puede pedir confirmación por email. Revisar el correo y confirmar.

Paso 5 — Verificar la conexión

El último paso es confirmar que tu PC puede comunicarse correctamente con GitHub usando la clave SSH que configuraste.

Ejecutar en Git Bash

```
ssh -T git@github.com
```

Primera conexión — mensaje de confianza

La primera vez que te conectas a GitHub, aparece este mensaje de seguridad:

```
The authenticity of host 'github.com' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhb...
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Escribí `yes` y presiona Enter. Esto solo pasa la primera vez.

Respuesta esperada

```
Hi tuusuario! You've successfully authenticated, but GitHub does not provide
shell access.
```

> **LISTO:** Si ves ese mensaje con tu usuario de GitHub, la configuración esta completa. Tu PC y GitHub ya se reconocen mutuamente.

Si aparece otro mensaje

Mensaje	Que significa
`Permission denied (publickey)`	La clave no está bien agregada en GitHub. Repetir el paso 4.
`ssh: connect to host github.com port 22`	Problema de red o firewall. Intentar desde otra red.
`Warning: Permanently added github.com`	Normal. Solo avisa que guardo GitHub como servidor conocido.

Resumen — todos los comandos

Para repasar o hacerlo de nuevo desde cero:

```
# 1. Verificar instalacion de Git
git --version

# 2. Configurar identidad
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
git config --global core.editor "code --wait"

# 3. Generar clave SSH (presionar Enter en todo)
ssh-keygen -t ed25519 -C "tu@email.com"

# 4. Verificar que se crearon los archivos
ls ~/.ssh

# 5. Copiar la clave publica al portapapeles
cat ~/.ssh/id_ed25519.pub | clip

# >> Ir a GitHub → Settings → SSH keys → New SSH key → pegar → guardar

# 6. Verificar la conexion con GitHub
ssh -T git@github.com
# Escribir 'yes' si pregunta por la autenticidad del host
```